

ARCHITECTURE

Architecture — CLI Agents ↔ Helix LLM Surfaces

Field	Value
Revision	2
Created	2026-09-05
Last modified	2026-09-07
Status	ACTIVE — arrows into HelixAgent were re-drawn on 2026-09-07 after end-to-end agent runs; revision 1 drew all three as "not probed"
Status summary	Shows which of the four CLI coding agents can reach which of the four Helix LLM surfaces, where auth/TLS boundaries sit, and where the tool-calling capability actually differs.
Scope	Companion to README.md and FAQ.md in this same directory
Authority	Every edge and label traces to a probe or cited vendor doc in docs/research/cli_agent_config_schemas/EVIDENCE.md, or to an end-to-end agent run in docs/qa/cli_agent_model_usability_20260907T113000Z/. This diagram makes no claim beyond what that evidence supports.

Table of contents

1. [How to read this diagram](#)
2. [The diagram](#)
3. [Reading the boundaries](#)

4. What the diagram deliberately leaves out

1. How to read this diagram

The left column is the four CLI coding agents; the right column is the four Helix LLM surfaces. An edge means "this agent can be configured to send requests to this surface" — it does **not** mean an agent is currently configured that way on any given machine. Edge style and label carry the capability information:

- **Thick solid arrow (==>)** — a confirmed, live-probed capability. The label says exactly what was confirmed (tool-calling present or absent).
- **Dashed arrow (-.->)** — the transport is reachable, but the combination does **not** work end to end: either it was refused (the label says so) or it was never driven to a verdict (the label says UNTESTED). Do not infer capability from the arrow alone; read the label.
- **Dashed cross-edge (- -x)** — no path exists at all. The target is a note, not a surface, explaining the vendor-side reason.

Node fill color marks the surface's auth/TLS posture (see the legend inside the diagram and §3 below).

2. The diagram

```
graph LR
  subgraph AGENTS["CLI Coding Agents"]
    OC["opcode 1.18.29"]
    PI["pi 0.84.4"]
    CR["crush 0.91.2"]
    CC["Claude Code 2.1.261"]
  end

  subgraph SURFACES["Helix LLM Surfaces"]
    CODER["Coder (llama.cpp)<br/>http://127.0.0.1:18434/v1<br/>auth: none"]
    GW["Gateway (HelixLLM)<br/>https://127.0.0.1:8443/v1<br/>auth: none · self-signed TLS"]
    HA["HelixAgent<br/>http://127.0.0.1:7061/v1<br/>auth: none"]
    HC["HelixCode<br/>http://127.0.0.1:8080<br/>auth: API key required"]
  end
```

```

OC ==>|"tool_calls: NO<br/>(fenced JSON in text)"| CODER
PI ==>|"tool_calls: NO<br/>(fenced JSON in text)"| CODER
CR ==>|"tool_calls: NO<br/>(fenced JSON in text)"| CODER

OC ==>|"tool_calls: YES<br/>(gateway translates)"| GW
PI ==>|"tool_calls: YES<br/>(gateway translates)"| GW
CR ==>|"tool_calls: YES<br/>(gateway translates)"| GW

OC -->|"array content rejected 400<br/>(pure sends strings:
UNTESTED)"| HA
PI -->|"BLOCKED: array content rejected 400"| HA
CR ==>|"WORKS end to end<br/>tool_calls: YES · ctx >= 200,046"|
HA

CC -->|"UNDER VERIFICATION<br/>Anthropic /v1/messages + API
key"| HC

CC --x NOTE["No OpenAI-compatible provider<br/>mechanism exists
in Claude Code<br/>(vendor constraint – see §3 below)"]

classDef noauth fill:#e8f5e9,stroke:#2e7d32,color:#1b1b1b;
classDef tlsauth fill:#fff8e1,stroke:#f9a825,color:#1b1b1b;
classDef keyauth fill:#ffebee,stroke:#c62828,color:#1b1b1b;
classDef note fill:#eceff1,stroke:#607d8b,color:#1b1b1b,stroke-
dasharray: 4 2;

class CODER,HA noauth;
class GW tlsauth;
class HC keyauth;
class NOTE note;

```

Node color legend: green = no auth required; amber = no auth required but self-signed TLS must be trusted; red = API key required; grey/dashed = an explanatory note, not a live surface.

3. Reading the boundaries

- **opencode, pi, and crush all reach Coder, Gateway, and HelixAgent** through the same mechanism — each agent's own custom "OpenAI-compatible provider" feature. This is the whole reason these three agents are grouped together in this guide: same mechanism, three different config file shapes (see [README.md §4](#)).
- **Coder vs. Gateway is the tool-calling boundary**, not an auth or TLS boundary — both surfaces require no API key. The difference is purely in how each surface shapes its response: Coder returns tool-call intent as a fenced JSON blob inside the assistant's plain-text message (which no agent's tool-call parser recognizes), while

Gateway translates the same underlying model's output into a real, structured `tool_calls` array with `finish_reason: "tool_calls"`. Same model, two different response shapes, because of what sits between the model and the client on each path.

- **Gateway is also the TLS boundary.** It is the only one of the four surfaces served over HTTPS with a self-signed certificate — every agent reaching it needs that certificate trusted first (`SSL_CERT_FILE` for `crush`; `NODE_EXTRA_CA_CERTS` / `NODE_TLS_REJECT_UNAUTHORIZED` for Node/Bun-based agents in principle, though a full live proof for `opencode` and `pi` against the Gateway was inconclusive — see [README.md §5](#)).
 - **HelixCode is the auth boundary.** It is the only surface of the four that rejects a request outright (HTTP 401) without an API key.
 - **Claude Code sits entirely outside the OpenAI-compatible-provider mechanism.** This is not a Helix limitation — it is a documented vendor constraint. Claude Code's own official documentation for the LLM Gateway Protocol (fetched 2026-09-05, <https://code.claude.com/docs/en/llm-gateway-protocol>) lists exactly three client-selectable API formats — Anthropic Messages, Amazon Bedrock InvokeModel, Google Agent Platform `rawPredict` — with no OpenAI Chat Completions option, and the companion LLM Gateway page states outright that Anthropic "doesn't support routing Claude Code to non-Claude models through any gateway." Claude Code's optional model-discovery step additionally filters by name, keeping only ids containing `claude` or `anthropic`. The only candidate path into Helix from Claude Code is HelixCode's Anthropic-shaped `/v1/messages` endpoint on `:8080` — drawn here as **under verification**, because a separate work stream was still establishing whether it actually works at the time this diagram was produced.
-

4. What the diagram deliberately leaves out

- **HelixAgent is now measured, and it is the only surface with room for a real agent prompt** (2026-09-07, superseding revision 1's "reachable, not probed"). It emits structured `tool_calls`, and it accepted a 200,046-token prompt while correctly recalling a marker planted at the very front of it — so it neither refused nor silently truncated. `crush` drives it end to end, including real tool use. `pi` and `opencode` cannot reach it: its OpenAI adapter returns HTTP 400 on

array-form content (cannot unmarshal array into Go struct field `OpenAIMessage.messages.content` of type string), which is the shape both send. See [README.md §3](#).

- **The Coder/Gateway boundary is not the tool-calling win it looks like.** The Gateway does translate tool calls — and it *also* silently discards array-form message content, returning 200 OK with a fluent answer to a question the model never saw (measured: 903 prompt tokens, including the entire instruction, dropped). It is safe only for plain-string senders like `crush`. See the warning in [README.md §2](#).
- **Both llama.cpp surfaces are capped at 32,768 tokens**, which is the model's `n_ctx_train` and cannot be raised. Every agent's baseline system prompt on this host exceeds it (`pi` 86,411, `crush` 96,564, `opcode` 134,043) because ~950 installed skills are inlined into it. That, not the arrows, is what decides usability.
- **Whether an agent is currently configured this way on any particular machine.** This diagram shows what is *possible* given each agent's own provider mechanism and each surface's own behavior — it is not a snapshot of any one operator's live config. Verify your own machine with the commands in [README.md §6](#).
- **HelixCode's `/v1/chat/completions` shape** is mentioned in [README.md §2](#) but not drawn separately here, since no agent in this guide has a confirmed path to either of HelixCode's two endpoint shapes yet.